```python
#Chaitanya Kharpate 23070521037
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

from sklearn.metrics import mean_squared_error, r2_score

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```python
#Load Dataset
df = pd.read_csv("Housing.csv")
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking
0	13300000	7420	4	2	3	yes	no	no	no	yes	2
1	12250000	8960	4	4	4	yes	no	no	no	yes	3
2	12250000	9960	3	2	2	yes	no	yes	no	no	2
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2

```python
#Split Features and Target
X = df.drop('price', axis=1)
y = df['price']
```

```python
#Identify Numerical and Categorical Columns
numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
categorical_features = X.select_dtypes(include=['object']).columns
```

```python
#Preprocessing
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)
```

```python
#Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```python
#Transform Data
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)
```

```python
#Create Model Function
def create_model(hidden_layers=2,
                 neurons=64,
                 activation='relu',
                 optimizer='adam'):
```

```
    model = Sequential()

    model.add(Dense(neurons,
                    activation=activation,
                    input_shape=(X_train.shape[1],)))

    for i in range(hidden_layers-1):
        model.add(Dense(neurons,
                        activation=activation))

    model.add(Dense(1))

    model.compile(
        optimizer=optimizer,
        loss='mse',
        metrics=['mae']
    )

    return model
```

```
#Hyperparameter Experiments No. 1
model1 = create_model(
    hidden_layers=2,
    activation='relu',
    optimizer='adam'
)

history1 = model1.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    verbose=0
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
#Hyperparameter Experiments No. 2
model2 = create_model(
    hidden_layers=3,
    activation='relu',
    optimizer='adam'
)

history2 = model2.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=16,
    verbose=0
)
```

```
#Hyperparameter Experiments No. 3
model3 = create_model(
    hidden_layers=3,
    activation='tanh',
    optimizer='rmsprop'
)

history3 = model3.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    verbose=0
)
```

```
#Hyperparameter Experiments No. 4 (Different Lerarning rate)
optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0005
)

model4 = create_model(
    hidden_layers=4,
```

```
    activation='relu',
    optimizer=optimizer
)

history4 = model4.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=64,
    verbose=0
)
```

```
#Evaluate Models
models = [model1, model2, model3, model4]

for i, model in enumerate(models, start=1):

    pred = model.predict(X_test)

    rmse = np.sqrt(mean_squared_error(y_test, pred))

    r2 = r2_score(y_test, pred)

    print(f"\nModel {i}")
    print("RMSE:", rmse)
    print("R²:", r2)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step

Model 1
RMSE: 5306356.143560664
R²: -4.570684432983398
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step
WARNING:tensorflow:5 out of the last 9 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_

Model 2
RMSE: 1365576.3250437523
R²: 0.6310667991638184
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/stepWARNING:tensorflow:6 out of the last 12 calls to <function TensorFlowTrainer.make_predi
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step

Model 3
RMSE: 5489004.501329544
R²: -4.960777759552002
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step

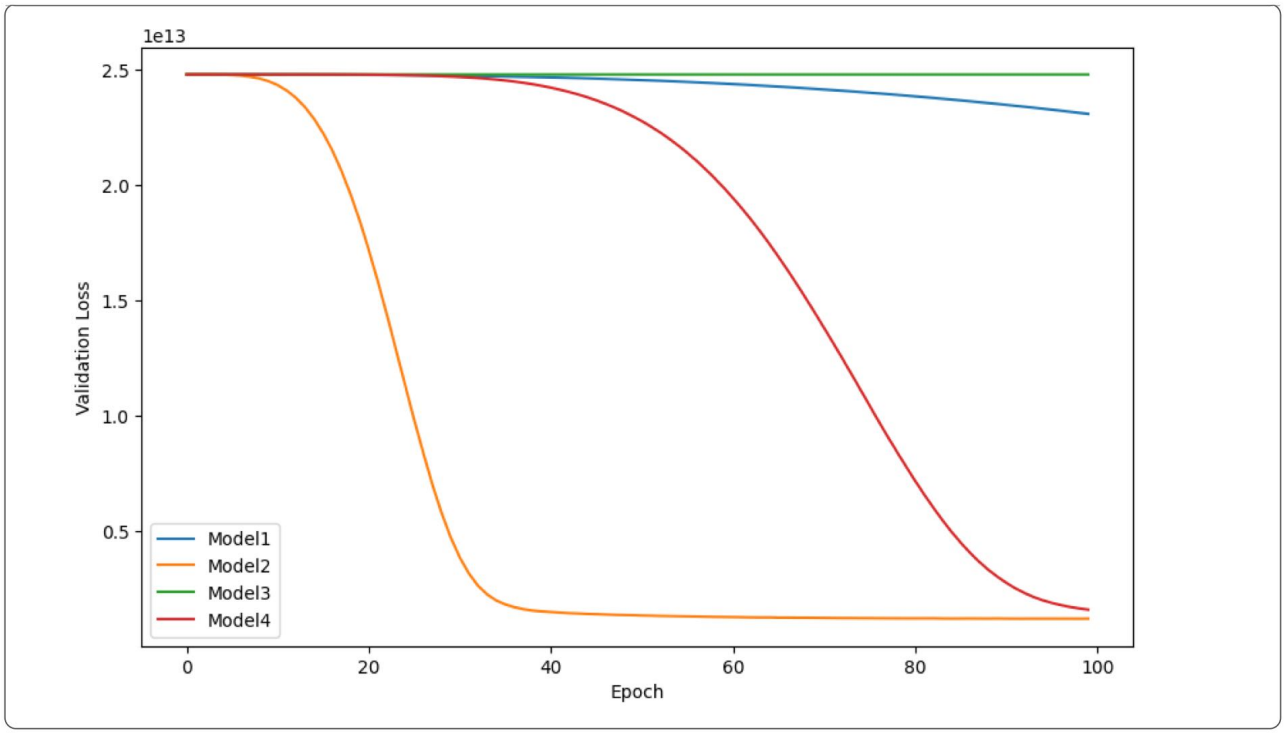
Model 4
RMSE: 1625719.2472798002
R²: 0.477114200592041
```

```
#Compare Validation Loss
plt.figure(figsize=(10,6))

plt.plot(history1.history['val_loss'],label='Model1')
plt.plot(history2.history['val_loss'],label='Model2')
plt.plot(history3.history['val_loss'],label='Model3')
plt.plot(history4.history['val_loss'],label='Model4')

plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.legend()

plt.show()
```


1e13
2.5
2.0
1.5
1.0
0.5
Validation Loss
Model1
Model2
Model3
Model4
0
20
40
60
80
100
Epoch